MI VDISP API

Version 2.08

REVISION HISTORY

Revision No.	Description	Date
2.03	• Initial release	04/12/2018
2.04	• Update statement	
2.05	• Update base on new architecture	10/29/2019
2.06	• Fix errors and format	01/13/2020
2.07	• Add API: MI_VDISP_InitDev and MI_VDISP_DeInitDev	04/26/2020
	• Add PROCFS Introduction	08/25/2021
2.08	• Add figure title • Fix hyperlinks • Add error code value	10/26/2021

TABLE OF CONTENTS

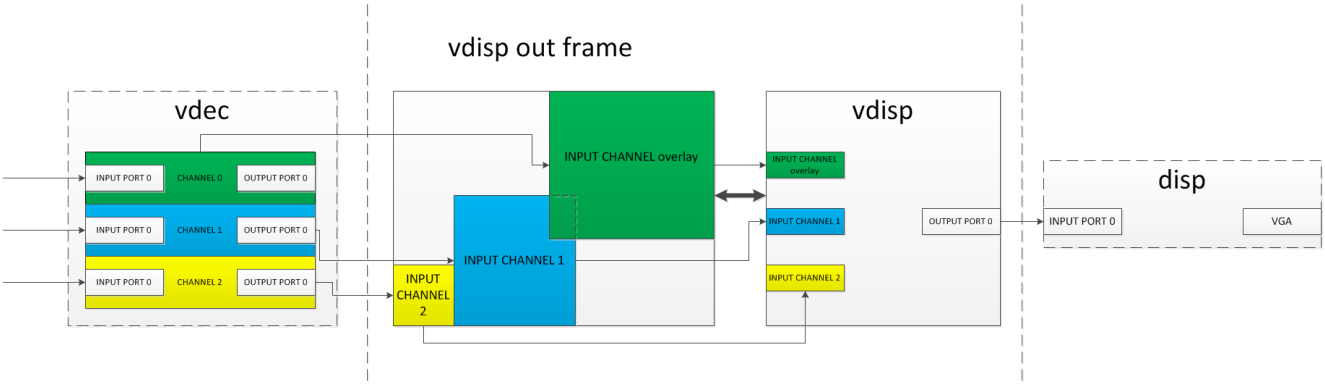
1. SUMMARY

1.1. Module Description

The module MI_VDISP is designed to assemble multiple outputs from front-end module along with user injections as needed to a whole data frame with thumbnail window for each input, and then output it.

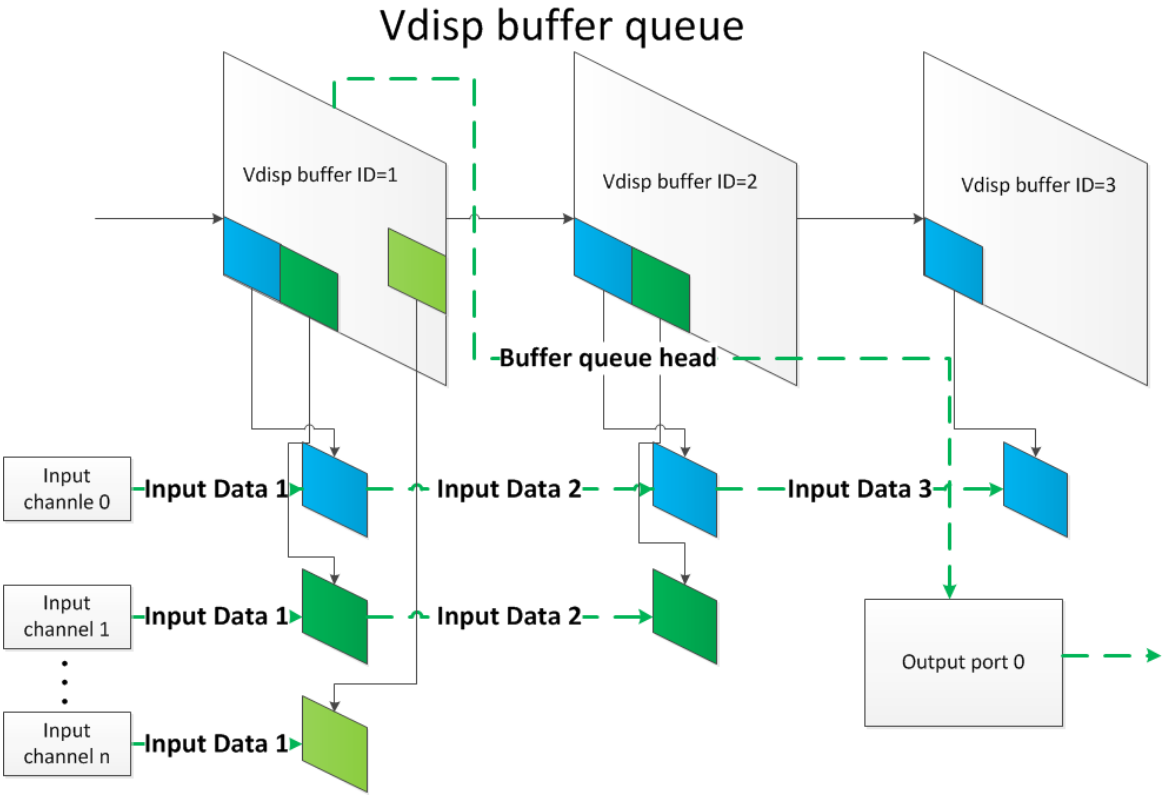
A typical scenario of VDISP goes like this:

- **a)** Assuming we have three vdec output channels: **green<0>**/**blue<1>**/**yellow<2>**
- **b)** Binding vdec **channel<0>** with vdisp input **channel<overlay>**, vdec **channel<1>** with vdisp input **channel<1>**, vdec **channel<2>** with vdisp input **channel<2>**
- **c)** Binding vdisp output port<0> with disp input port<0>
- **d)** Outputting data frame by VGA



1.2. Flow Chart

Unlike many other modules, VDISP is a PURE software module, no proprietary HW IP occupied. VDISP is a proxy of memory buffer which supplies output buffers of those which would eventually be sent to VDISP's input channels for its front end module by allocating buffers from its own output buffers maintained through a buffer queue.
Thus, we can assemble multiple inputs without copying anything.



1.3. Constraints

a) Alignment for beginning horizontal coordinate (u32OutX) of input channels is essential, because HW IPs were designed to access aligned memory address usually. The specific alignment value is determined by front-end modules. Generally, 16 is highly recommended, although 8 is sufficient enough for some scenarios.

b) Currently, VDISP supports two input data formats:
- E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420
- E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422

c) Currently, VDISP only supports fixed frame-rate output. See parameter u32FrmRate of MI_VDISP_OutputPortAttr_t.

d) Currently, VDISP does not support **format-conversion/cropping/scaling**.

e) Please refer to SPECIFICATION for more details.

2. LIST OF APIS

Name of API	Function
MI_VDISP_Init	Initialize Module
MI_VDISP_Exit	De-initialize Module
MI_VDISP_OpenDevice	Open a virtual VDISP device
MI_VDISP_CloseDevice	Close a virtual VDISP device
MI_VDISP_SetOutputPortAttr	Set attributes of output port
MI_VDISP_GetOutputPortAttr	Get attributes of output port
MI_VDISP_SetInputChannelAttr	Set attributes of input channel
MI_VDISP_GetInputChannelAttr	Get attributes of input channel
MI_VDISP_EnableInputChannel	Enable an input channel
MI_VDISP_DisableInputChannel	Disable an input channel
MI_VDISP_StartDev	Start a VDISP device
MI_VDISP_StopDev	Stop a VDISP device
MI_VDISP_InitDev	Initialize VDISP device
MI_VDISP_DeInitDev	De-initialize VDISP device

2.1. MI_VDISP_Init

- Function

  Initialize VDISP module.

- Syntax

  MI_S32 MI_VDISP_ModuleInit(void);

- Parameter

  None

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  None

- Example

  None

- Reference

  MI_VDISP_Exit

2.2. MI_VDISP_Exit

- Function

  De-initialize VDISP module.

- Syntax

  MI_S32 MI_VDISP_ModuleDeinit(void);

- Parameter

  None

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  None.

- Example

  None.

- Reference

  MI_VDISP_Init

2.3. MI_VDISP_OpenDevice

- Function

  Open a VDISP virtual device.

- Syntax

  MI_S32 MI_VDISP_OpenDevice(MI_VDISP_DEV DevId)

- Parameter

Name	Description	Input/Output
DevId	Device ID	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  Initialize VDISP before opening a device.

- Example

  None.

- Reference

  None.

2.4. MI_VDISP_CloseDevice

- Function

  Close a VDISP virtual device.

- Syntax

  MI_S32 MI_VDISP_CloseDevice(MI_VDISP_DEV DevId);

- Parameter

Name	Description	Input/Output
DevId	Device ID	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  None.

- Example

  None.

- Reference

  None.

2.5. MI_VDISP_SetOutputPortAttr

- Function

  Set attributes of output port.

- Syntax

  MI_S32 MI_VDISP_SetOutputPortAttr(MI_VDISP_DEV DevId, MI_VDISP_PORT PortId, MI_VDISP_OutputPortAttr_t *pstOutputPortAttr);

- Parameter

Name	Description	Input/Output
DevId	Device ID	Input
PortId	Port ID	Input
pstOutputPortAttr	Pointer to attributes of output port	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  None.

- Example

  None.

- Reference

  None.

2.6. MI_VDISP_GetOutputPortAttr

- Function

  Get attributes of output port.

- Syntax

  MI_S32 MI_VDISP_GetOutputPortAttr(MI_VDISP_DEV DevId, MI_VDISP_PORT PortId, MI_VDISP_OutputPortAttr_t *pstOutputPortAttr);

- Parameter

Name	Description	Input/Output
DevId	Device ID	Input
PortId	Port ID	Input
pstOutputPortAttr	Pointer to attributes of output port	Output

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  None.

- Example

  None.

- Reference

  None.

2.7. MI_VDISP_SetInputChannelAttr

- Function

  Set attributes of input channel.

- Syntax

  MI_S32 MI_VDISP_SetInputChannelAttr(MI_VDISP_DEV DevId, MI_VDISP_CHN ChnId,MI_VDISP_InputChnAttr_t *pstInputChnAttr);

- Parameter

Name	Description	Input/Output
DevId	Device ID	Input
ChnId	Channel Id	Input
pstInputChntAttr	Pointer to attributes of input channel	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  None.

- Example

  None.

- Reference

  None.

2.8. MI_VDISP_GetInputChannelAttr

- Function

  Get attributes of input channel.

- Syntax

  MI_S32 MI_VDISP_GetInputChannelAttr(MI_VDISP_DEV DevId, MI_VDISP_CHN ChnId,MI_VDISP_InputChnAttr_t *pstInputChnAttr);

- Parameter

Name	Description	Input/Output
DevId	Device ID	Input
ChnId	Channel Id	Input
pstInputChnAttr	Pointer to attributes of input channel	Output

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details
- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)
- Note

  None.
- Example

  None.
- Reference

  None.

2.9. MI_VDISP_EnableInputChannel

- Function

  Enable an input channel.
- Syntax

  MI_S32 MI_VDISP_EnableInputChannel(MI_VDISP_DEV DevId, MI_VDISP_CHN ChnId);
- Parameter

Name	Description	Input/Output
DevId	Device ID	Input
ChnId	Channel ID	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details
- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)
- Note

  None.
- Example

  None.
- Reference

  None.

2.10. MI_VDISP_DisableInputChannel

- Function

  Disable an input channel.

- Syntax

  MI_S32 MI_VDISP_DisableInputChannel(MI_VDISP_DEV DevId, MI_VDISP_CHN ChnId);

- Parameter

Name	Description	Input/Output
DevId	Device ID	Input
ChnId	Channel ID	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  None.

- Example

  None.

- Reference

  None.

2.11. MI_VDISP_StartDev

- Function

  Start a VDISP device.

- Syntax

  MI_S32 MI_VDISP_StartDev(MI_VDISP_DEV DevId);

- Parameter

Name	Description	Input/Output
DevId	Device ID	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  None.

- Function

  None.

- Reference

  None.

2.12. MI_VDISP_StopDev

- Function

  Stop a VDISP device.

- Syntax

  MI_S32 MI_VDISP_StopDev(MI_VDISP_DEV DevId);

- Parameter

Name	Description	Input/Output
DevId	Device ID	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Dependency
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note

  Frame data buffered in VDISP will be cleaned after device is stopped.

- Example

  None.

- Reference

  None.

2.13. MI_VDISP_InitDev

- Function

  Initialize VDISP device.

- Syntax

  MI_S32 MI_VDISP_InitDev(MI_VDISP_InitParam_t *pstInitParam);

- Parameters

Parameter Name	Description	Input/Output
pstInitParam	Initialization Parameter	Input

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Requirement
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note
  - pstInitParam is unused for now; pass NULL instead.
  - For version 2.07 and above, we recommend using this interface to replace the original MI_VDISP_Init interface.

- Example

  N/A

- Related API

  N/A

2.14. MI_VDISP_DeInitDev

- Function

  De-initialize VDISP device.

- Syntax

  MI_S32 MI_VDISP_DeInitDev(void);

- Parameters

  N/A

- Return Value
  - Zero: Successful
  - Non-zero: Failed, see error code for details

- Requirement
  - Header: mi_vdisp.h mi_vdisp_datatype.h
  - Library: libmi_vdisp.a(so)

- Note
  - For Version 2.07 and above, we recommend using this interface to replace the original MI_VDISP_Exit interface.

- Example

  N/A

- Related API

  N/A

3. DATA TYPE

Definition of data type in MI VDISP is as listed in the following table:

Data Type	Description
MI_VDISP_DEV	Type of device ID
MI_VDISP_PORT	Type of port ID
MI_VDISP_CHN	Type of channel ID
MI_VDISP_OutputPortAttr_t	Type of output port attributes
MI_VDISP_InputChnAttr_t	Type of input channel attributes
MI_VDISP_InitParam_t	Type of initialization parameter of VDISP device

3.1. MI_VDISP_DEV

- Description

  Type of device ID.

- Definition

  typedef MI_S32 MI_VDISP_DEV;

- Member

  None.

- Note

  Valid for natural number.

- Reference

  None.

3.2. MI_VDISP_PORT

- Description

  Type of port ID.

- Definition

  typedef MI_S32 MI_VDISP_PORT;

- Member

  None.

- Note

  Valid for natural number.

- Reference

  None.

3.3. MI_VDISP_CHN

- Description

  Type of channel ID.

- Definition

  typedef MI_S32 MI_VDISP_CHN;

- Member

  None.

- Note

  Valid for natural number.

- Reference

  None.

3.4. MI_VDISP_OutputPortAttr_t

- Description

  Type of output port attributes.

- Definition

```
typedef struct MI_VDISP_OutputPortAttr_s
{
    MI_U32 u32BgColor;    /* Background color of a output port, in YUV format. [23:16]:v, [15:8]:y, [7:0]:u*/
    PIXEL_FORMAT_E ePixelFormat; /* pixel format of a output port */
    MI_U64 u64pts; /* current PTS */
    MI_U32 u32FrmRate; /* the frame rate of output port */
    MI_U32 u32Width; /* the frame width of a output port */
    MI_U32 u32Height; /* the frame height of a output port */
} MI_VDISP_OutputPortAttr_t;
```

- Member

Name	Description
u32BgColor	Background color of output frame, yuv444 [23:16]: v [15:8]: y [7:0]: u
ePixelFormat	Pixel format of output frame.
u64pts	Present PTS of output frame.
u32FrmRate	Fixed output frame-rate 0: Use frame-rate value in binding setting
u32Width	Width of output frame
u32Height	Height of output frame

- Note

  None.

- Reference

  None.

3.5. MI_VDISP_InputChnAttr_t

- Description

  Type of input channel attributes.

- Definition

```
typedef struct MI_VDISP_InputPortAttr_s
{
   MI_U32 u32OutX; /* the output frame X position of this input port */
   MI_U32 u32OutY; /* the output frame Y position of this input port */
   MI_U32 u32OutWidth; /* the output frame width of this input port */
MI_U32 u32OutHeight; /* the output frame height of this input port */
MI_S32 s32IsFreeRun; /* is this port free run */
} MI_VDISP_InputPortAttr_t;
```

- Member

Name	Description
u32OutX	Coordinate X where thumbnail of input frame will be present in assembled output frame
u32OutY	Coordinate Y where thumbnail of input frame will be present in assembled output frame
u32OutWidth	Width for thumbnail of input frame which will be present in the assembled output frame
u32OutHeight	Height for thumbnail of input frame which will be present in the assembled output frame
s32IsFreeRun	0: Input channel's frame in which PTS smaller than present PTS of output port will be ignored. Other: Input frame will not be checked

- Note

  None.

- Reference

  None.

3.6. MI_VDISP_InitParam_t

- Description

  VDISP device initialization parameter.

- Definition

```
typedef struct MI_VDISP_InitParam_s
{
    MI_U32 u32DevId;
    MI_U8 *u8Data;
} MI_VDISP_InitParam_t;
```

- Member

Member	Description
u32DevId	Device ID
u8Data	Data pointer

- Note

N/A.

- Related Data Type and Interface

MI_VDISP_InitDev

4. ERROR CODE

The VDISP error codes are as shown in Table 4-1:

Table 4-1: API Error Code

Error Code	Definition	Description
0XA0142000	MI_SUCCESS	Success
0XA0142031	MI_VDISP_ERR_FAIL	Operation failed, please refer to system log
0XA0142001	MI_VDISP_ERR_INVALID_DEVID	Illegal device id, please refer to system log
0XA0142003	MI_VDISP_ERR_ILLEGAL_PARAM	Illegal parameter, please refer to system log
0XA0142008	MI_VDISP_ERR_NOT_SUPPORT	Operation failed, output color format was not supported
0XA0142022	MI_VDISP_ERR_MOD_INITED	Operation failed, VDISP module already initialized
0XA0142021	MI_VDISP_ERR_MOD_NOT_INIT	Operation failed, VDISP module needs to be initialized
0XA0142240	MI_VDISP_ERR_DEV_OPENED	Operation failed, VDISP device already opened
0XA0142241	MI_VDISP_ERR_DEV_NOT_OPEN	Operation failed, open VDISP device first
0XA0142027	MI_VDISP_ERR_DEV_NOT_STOP	Operation failed, stop VDISP device first
0XA0142242	MI_VDISP_ERR_DEV_NOT_CLOSE	Operation failed, close VDISP device first
0XA0142007	MI_VDISP_ERR_NOT_CONFIG	Operation failed, configure VDISP output port first
0XA0142024	MI_VDISP_ERR_PORT_NOT_DISABLE	Operation failed, disable input channels first
0XA0142243	MI_VDISP_ERR_PORT_NOT_UNBIND	Operation failed, unbind input/output first

5. SPECIFICATION

Table 5-1: Specification of VDISP

Definition	Value	Description
VDISP_MAX_DEVICE_NUM	4	Max virtual device number
VDISP_MAX_CHN_NUM_PER_DEV	16	Max non-overlay input channel number per device
VDISP_MAX_INPUTPORT_NUM	1	Max port number of input channel.
VDISP_MAX_OVERLAYINPUTCHN_NUM	4	Max overlay input channel number per device. Built-in constable value. Rebuilding module is required if more overlay channels are needed.
VDISP_OVERLAYINPUTCHNID	VDISP_MAX_CHN_NUM_PER_DEV	Starting ID of overlay input channel
VDISP_MAX_OUTPUTPORT_NUM	1	Max output port number per device

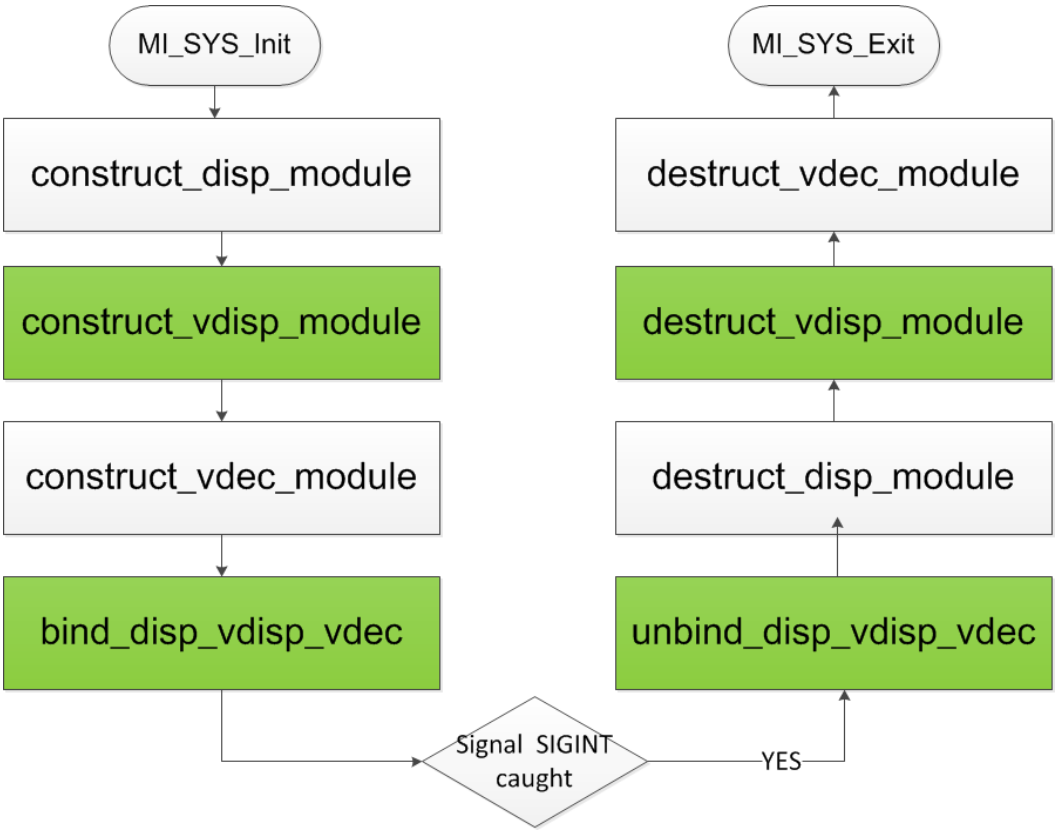
6. APPENDIX

6.1. Sample

This section details an implementation of the scenario described in Summary section.

6.1.1 Flow Chart of Sample

GREEN parts indicate the module vdisp was involved.



6.1.2 Partial Sample Involving vdisp

6.1.2.1. Construct module vdisp

```
/*
 * Initialize module vdisp
 *      v
 * Open vdisp device
 *      v
 * Setting parameters of vdisp input channel/output port
 *      v
 * Activate input channels
 *      v
 * Start vdisp device
 */
void construct_vdisp_module(void)
{
#define MAKE_YUYV_VALUE(y,u,v)    ((y) << 24) | ((u) << 16) | ((y) << 8) | (v)
#define YUYV_BLACK                MAKE_YUYV_VALUE(0,128,128)
#define ALIGN16_DOWN(x) (x&0xFFF0)

    typedef struct RECT
    {
        /*
        (x,y)_ _ _ _ _ _ _
          |             |
          |           (h)
          |_ _ _(w)_ _ _|
        */
        short x;
        short y;
        short w;
        short h;
    } RECT;

    /*
     * Initialize module vdisp
     */
    MI_VDISP_Init();

    MI_VDISP_InputChnAttr_t stInputChnAttr;
    MI_VDISP_OutputPortAttr_t stOutputPortAttr;
    MI_VDISP_DEV DevId = 0;
    RECT rect_0, rect_1, rect_2;
    MI_VDISP_CHN Chn_0_Id = VDISP_OVERLAYINPUTCHNID;
    MI_VDISP_CHN Chn_1_Id = 1;
    MI_VDISP_CHN Chn_2_Id = 2;

    /*
```

```
 * open a virtual device of vdisp
 */
MI_VDISP_OpenDevice(DevId);

/*
 * align starting horizontal corrdinate by 16
 * setting parameters of input channel
 */
rect_0.x = ALIGN16_DOWN(960);
rect_0.y = 0;
rect_0.w = 960;
rect_0.h = 960;
stInputChnAttr.s32IsFreeRun = TRUE;
stInputChnAttr.u32OutHeight = rect_0.h;
stInputChnAttr.u32OutWidth = rect_0.w;
stInputChnAttr.u32OutX = rect_0.x;
stInputChnAttr.u32OutY = rect_0.y;
/*
 *setting parameters of input channel<VDISP_OVERLAYINPUTCHNID>
 */
MI_VDISP_SetInputChannelAttr(DevId, Chn_0_Id, &stInputChnAttr);

/*
 * align starting horizontal corrdinate by 16
 * setting parameters of input channel
 */
rect_1.x = ALIGN16_DOWN(0);
rect_1.y = 1080 - 300;
rect_1.w = ALIGN16_DOWN(300);
rect_1.h = 300;
stInputChnAttr.s32IsFreeRun = TRUE;
stInputChnAttr.u32OutHeight = rect_1.h;
stInputChnAttr.u32OutWidth = rect_1.w;
stInputChnAttr.u32OutX = rect_1.x;
stInputChnAttr.u32OutY = rect_1.y;
/*
 *setting parameters of input channel<1>
 */
MI_VDISP_SetInputChannelAttr(DevId, Chn_1_Id, &stInputChnAttr);

/*
 * align starting horizontal corrdinate by 16
 * setting parameters of input channel
 */
rect_2.x = ALIGN16_DOWN(300);
rect_2.y = 1080 - 700;
```

```
    rect_2.w = 700;
    rect_2.h = 700;
    stInputChnAttr.s32IsFreeRun = TRUE;
    stInputChnAttr.u32OutHeight = rect_2.h;
    stInputChnAttr.u32OutWidth = rect_2.w;
    stInputChnAttr.u32OutX = rect_2.x;
    stInputChnAttr.u32OutY = rect_2.y;
    /*
     *setting parameters of input channel<2>
     */
    MI_VDISP_SetInputChannelAttr(DevId, Chn_2_Id, &stInputChnAttr);

    //setting output color format
    stOutputPortAttr.ePixelFormat = E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420;
    //setting backgroud color of vdisp's output frame
    stOutputPortAttr.u32BgColor = YUYV_BLACK;
    //setting output framerate
    stOutputPortAttr.u32FrmRate = 30;
    //setting height of output frame
    stOutputPortAttr.u32Height = 1080;
    //setting width of output frame
    stOutputPortAttr.u32Width = 1920;
    //settin initial pts of output frame
    stOutputPortAttr.u64pts = 0;

    /*
     * setting parameters of output port<0>
     */
    MI_VDISP_SetOutputPortAttr(DevId,0,&stOutputPortAttr);

    /*
     * activating input channels <VDISP_OVERLAYINPUTCHNID,1,2>
     */
    MI_VDISP_EnableInputChannel(DevId, Chn_0_Id);
    MI_VDISP_EnableInputChannel(DevId, Chn_1_Id);
    MI_VDISP_EnableInputChannel(DevId, Chn_2_Id);

    /*
     * start vdisp device
     */
    MI_VDISP_StartDev(DevId);
}
```

6.1.2.2. Binding front/back ends

```
/*
 * binding vdec out0 -> vdisp inOverlay---\
 * binding vdec out1 -> vdisp in1 --------->vdisp out0--> disp in0
 * binding vdec out2 -> vdisp in2---------/
 */
void bind_disp_vdisp_vdec(void)
{
    MI_SYS_ChnPort_t stSrcChnPort;
    MI_SYS_ChnPort_t stDstChnPort;

    /*
     * binding vdisp out0-> disp in0
     * <chn,dev,port> : (0,0,0) -> (0,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_DISP;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

    /*
     * binding vdec out0-> vdisp inOverlay
     * <chn,dev,port> : (0,0,0) -> (VDISP_OVERLAYINPUTCHNID,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = VDISP_OVERLAYINPUTCHNID;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

    /*
     * binding vdec out0-> vdisp in1
     * <chn,dev,port> : (1,0,0) -> (1,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
```

```
    stSrcChnPort.u32ChnId = 1;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 1;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

    /*
     * binding vdec out2-> vdisp in2
     * <chn,dev,port> : (2,0,0) -> (2,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 2;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 2;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

}
```

6.1.2.3. Unbinding front/back ends

The procedure is quite similar to binding front/back ends. Please refer to Full implementation for actual source code.

6.1.2.4. Destruct module vdisp

```
/*
 * disable all channels enabled
 *      v
 * stop vdisp device
 *      v
 * close vdisp device
 *      v
 * exit vdisp
 */
void destruct_vdisp_module(void)
{
    MI_VDISP_DEV DevId = 0;
    MI_VDISP_CHN Chn_0_Id = VDISP_OVERLAYINPUTCHNID;
```

```
    MI_VDISP_CHN Chn_1_Id = 1;
    MI_VDISP_CHN Chn_2_Id = 2;
   /*
    * diable vdisp channels <VDISP_OVERLAYINPUTCHNID,1,2>
    */
    MI_VDISP_DisableInputChannel(DevId, Chn_0_Id);
    MI_VDISP_DisableInputChannel(DevId, Chn_1_Id);
    MI_VDISP_DisableInputChannel(DevId, Chn_2_Id);

   /*
    *stop vdisp device
    */
    MI_VDISP_StopDev(DevId);
   /*
    *close vdisp device
    */
    MI_VDISP_CloseDevice(DevId);
   /*
    *exit vdisp
    */
    MI_VDISP_Exit();
}
```

6.1.3 Full implementation

```
#include <threads.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <assert.h>
#include <signal.h>


#include <mi_sys_datatype.h>
#include <mi_sys.h>
#include <mi_vdec_datatype.h>
#include <mi_vdec.h>
#include <mi_disp_datatype.h>
#include <mi_disp.h>
#include <mi_vdisp_datatype.h>
#include <mi_vdisp.h>



static volatile int keepRunning = 1;
```

```
static void intHandler(int dummy)
{
    keepRunning = 0;
}

void construct_disp_module(void)
{

    MI_DISP_PubAttr_t stDispPubAttr;
    stDispPubAttr.eIntfSync = E_MI_DISP_OUTPUT_1080P60;
    stDispPubAttr.eIntfType = E_MI_DISP_INTF_VGA;
    MI_DISP_SetPubAttr(0, &stDispPubAttr);

    MI_DISP_InputPortAttr_t stInputPortAttr;
    stInputPortAttr.u16SrcWidth = 1920;
    stInputPortAttr.u16SrcHeight = 1080;
    stInputPortAttr.stDispWin.u16X = 0;
    stInputPortAttr.stDispWin.u16Y = 0;
    stInputPortAttr.stDispWin.u16Width = 1920;
    stInputPortAttr.stDispWin.u16Height = 1080;
    MI_DISP_SetInputPortAttr(0, 0, &stInputPortAttr);

    MI_DISP_Enable(0);
    MI_DISP_EnableVideoLayer(0);
    MI_DISP_EnableInputPort(0, 0);
}
void destruct_disp_module(void)
{
    MI_DISP_DisableInputPort(0, 0);
    MI_DISP_DisableVideoLayer(0);
    MI_DISP_UnBindVideoLayer(0, 0);
    MI_DISP_Disable(0);
}
typedef struct vdecStream
{
    FILE *fp;
    MI_VDEC_CHN VdecChn;
    int frameRate;
} vdecStream;

MI_U64 get_pts(MI_U32 u32FrameRate)
{
    if(0 == u32FrameRate)
    {
        return (MI_U64)(-1);
    }
```

```
    return (MI_U64)(1000 / u32FrameRate);
}

int push_stream(void *args)
{

#define MI_U32VALUE(pu8Data, index) (pu8Data[index]<<24)|(pu8Data[index+1]<<16)|
(pu8Data[index+2]<<8)|(pu8Data[index+3])
#define VESFILE_READER_BATCH (128 * 1024)

    MI_S32 s32Ret = MI_SUCCESS;
    MI_VDEC_CHN VdecChn;
    MI_U32 u32FrmRate = 0;
    MI_U8 *pu8Buf = NULL;
    MI_U32 u32Len = 0;
    MI_U32 u32FrameLen = 0;
    MI_U64 u64Pts = 0;
    MI_U8 au8Header[16] = {0};
    MI_U32 u32Pos = 0;
    MI_VDEC_ChnStat_t stChnStat;
    vdecStream *vdecS = (vdecStream *)args;
    MI_VDEC_VideoStream_t stVdecStream;
    MI_S32 s32TimeOutMs = 20;

    MI_U32 u32FpBackLen = 0; // if send stream failed, file pointer back length

    VdecChn    = vdecS->VdecChn;
    u32FrmRate = vdecS->frameRate;

    pu8Buf = malloc(VESFILE_READER_BATCH);

    while(keepRunning)
    {
        ///frame mode, read one frame lenght every time

        memset(au8Header, 0, 16);
        u32Pos = fseek(vdecS->fp, 0L, SEEK_CUR);
        u32Len = fread(au8Header, 16, 1, vdecS->fp);

        if(u32Len <= 0)
        {
            fseek(vdecS->fp, 0, SEEK_SET);
            continue;
        }
```

```
        u32FrameLen = MI_U32VALUE(au8Header, 4);
        if(u32FrameLen > VESFILE_READER_BATCH)
        {
            fseek(vdecS->fp, 0, SEEK_SET);
            continue;
        }

        u32Len = fread(pu8Buf, u32FrameLen, 1, vdecS->fp);

        if(u32Len <= 0)
        {
            fseek(vdecS->fp, 0, SEEK_SET);
            continue;
        }

        stVdecStream.pu8Addr = pu8Buf;
        stVdecStream.u32Len = u32FrameLen;
        stVdecStream.u64PTS = u64Pts;
        stVdecStream.bEndOfFrame = 1;
        stVdecStream.bEndOfStream = 0;

        u32FpBackLen = stVdecStream.u32Len + 16; //back length

        if(MI_SUCCESS != (s32Ret = MI_VDEC_SendStream(VdecChn, &stVdecStream, s32Time
OutMs)))
        {
            fseek(vdecS->fp, - u32FpBackLen, SEEK_CUR);
        }

        u64Pts = u64Pts + get_pts(1000 / u32FrmRate);

        memset(&stChnStat, 0x0, sizeof(stChnStat));
        MI_VDEC_GetChnStat(VdecChn, &stChnStat);
        usleep(20 * 1000);

    }

    free(pu8Buf);
    return NULL;
}

thrd_t thr0, thr1, thr2;
vdecStream vS0, vS1, vS2;

void construct_vdec_module(void)
{
```

```
    MI_VDEC_ChnAttr_t stChnAttr;
    MI_VDEC_CHN Chn_0_Id = 0;
    MI_VDEC_CHN Chn_1_Id = 1;
    MI_VDEC_CHN Chn_2_Id = 2;

    memset(&stChnAttr, 0, sizeof(MI_VDEC_ChnAttr_t));

    MI_VDEC_InitParam_t stVdecInitParam;
    MI_VDEC_ChnParam_t stChnParam;
    MI_VDEC_OutputPortAttr_t stOutputPortAttr;
    stVdecInitParam.bDisableLowLatency = FALSE;
    MI_VDEC_InitDev(&stVdecInitParam);

    stChnAttr.eCodecType    = E_MI_VDEC_CODEC_TYPE_H264;
    stChnAttr.u32PicWidth   = 1920;
    stChnAttr.u32PicHeight  = 1080;
    stChnAttr.eVideoMode    = E_MI_VDEC_VIDEO_MODE_FRAME;
    stChnAttr.u32BufSize    = 1024 * 1024;
    stChnAttr.eDpbBufMode   = E_MI_VDEC_DPB_MODE_NORMAL;
    stChnAttr.stVdecVideoAttr.u32RefFrameNum = 2;
    stChnAttr.u32Priority = 0;


    MI_VDEC_CreateChn(Chn_0_Id, &stChnAttr);
    stOutputPortAttr.u16Width = 960;
    stOutputPortAttr.u16Height = 960;
    MI_VDEC_SetOutputPortAttr(Chn_0_Id, &stOutputPortAttr);

    MI_VDEC_CreateChn(Chn_1_Id, &stChnAttr);
    stOutputPortAttr.u16Width = 300;
    stOutputPortAttr.u16Height = 300;
    MI_VDEC_SetOutputPortAttr(Chn_1_Id, &stOutputPortAttr);

    MI_VDEC_CreateChn(Chn_2_Id, &stChnAttr);
    stOutputPortAttr.u16Width = 700;
    stOutputPortAttr.u16Height = 700;
    MI_VDEC_SetOutputPortAttr(Chn_2_Id, &stOutputPortAttr);

    MI_VDEC_StartChn(Chn_0_Id);
    MI_VDEC_StartChn(Chn_1_Id);
    MI_VDEC_StartChn(Chn_2_Id);


#define VDEC_FILE0 "./vdisp_demo0.h264"
#define VDEC_FILE1 "./vdisp_demo1.h264"
#define VDEC_FILE2 "./vdisp_demo2.h264"
```

```
    vS0.fp = fopen(VDEC_FILE0, "rb");
    vS0.frameRate = 30;
    vS0.VdecChn = Chn_0_Id;
    assert(vS0.fp);
    thrd_create(&thr0, push_stream, &vS0);
    vS1.fp = fopen(VDEC_FILE1, "rb");
    vS1.frameRate = 30;
    vS1.VdecChn = Chn_1_Id;
    assert(vS1.fp);
    thrd_create(&thr1, push_stream, &vS1);
    vS2.fp = fopen(VDEC_FILE2, "rb");
    vS2.frameRate = 30;
    vS2.VdecChn = Chn_2_Id;
    assert(vS2.fp);
    thrd_create(&thr2, push_stream, &vS2);
}
void destruct_vdec_module(void)
{
    thrd_join(thr0,NULL);
    thrd_join(thr1,NULL);
    thrd_join(thr2,NULL);
    MI_VDEC_StopChn(vS0.VdecChn);
    MI_VDEC_StopChn(vS1.VdecChn);
    MI_VDEC_StopChn(vS2.VdecChn);
    MI_VDEC_DestroyChn(vS0.VdecChn);
    MI_VDEC_DestroyChn(vS1.VdecChn);
    MI_VDEC_DestroyChn(vS2.VdecChn);
    MI_VDEC_DeInitDev();
}

/*
 * Initialize module vdisp
 *      v
 * Open vdisp device
 *      v
 * Setting parameters of vdisp input channel/output port
 *      v
 * Activate input channels
 *      v
 * Start vdisp device
 */
void construct_vdisp_module(void)
{
#define MAKE_YUYV_VALUE(y,u,v)    ((y) << 24) | ((u) << 16) | ((y) << 8) | (v)
#define YUYV_BLACK                MAKE_YUYV_VALUE(0,128,128)
#define ALIGN16_DOWN(x) (x&0xFFF0)
```

```
typedef struct RECT
{
    /*
    (x,y)_ _ _ _ _ _ _
      |             |
      |           (h)
      |_ _ _(w)_ _ _|
    */
    short x;
    short y;
    short w;
    short h;
} RECT;

/*
 * Initialize module vdisp
 */
MI_VDISP_Init();

MI_VDISP_InputChnAttr_t stInputChnAttr;
MI_VDISP_OutputPortAttr_t stOutputPortAttr;
MI_VDISP_DEV DevId = 0;
RECT rect_0, rect_1, rect_2;
MI_VDISP_CHN Chn_0_Id = VDISP_OVERLAYINPUTCHNID;
MI_VDISP_CHN Chn_1_Id = 1;
MI_VDISP_CHN Chn_2_Id = 2;

/*
 * open a virtual device of vdisp
 */
MI_VDISP_OpenDevice(DevId);

/*
 * align starting horizontal corrdinate by 16
 * setting parameters of input channel
 */
rect_0.x = ALIGN16_DOWN(960);
rect_0.y = 0;
rect_0.w = 960;
rect_0.h = 960;
stInputChnAttr.s32IsFreeRun = TRUE;
stInputChnAttr.u32OutHeight = rect_0.h;
stInputChnAttr.u32OutWidth = rect_0.w;
stInputChnAttr.u32OutX = rect_0.x;
stInputChnAttr.u32OutY = rect_0.y;
```

```
    /*
     *setting parameters of input channel<VDISP_OVERLAYINPUTCHNID>
     */
    MI_VDISP_SetInputChannelAttr(DevId, Chn_0_Id, &stInputChnAttr);

    /*
     * align starting horizontal corrdinate by 16
     * setting parameters of input channel
     */
    rect_1.x = ALIGN16_DOWN(0);
    rect_1.y = 1080 - 300;
    rect_1.w = ALIGN16_DOWN(300);
    rect_1.h = 300;
    stInputChnAttr.s32IsFreeRun = TRUE;
    stInputChnAttr.u32OutHeight = rect_1.h;
    stInputChnAttr.u32OutWidth = rect_1.w;
    stInputChnAttr.u32OutX = rect_1.x;
    stInputChnAttr.u32OutY = rect_1.y;
    /*
     *setting parameters of input channel<1>
     */
    MI_VDISP_SetInputChannelAttr(DevId, Chn_1_Id, &stInputChnAttr);

    /*
     * align starting horizontal corrdinate by 16
     * setting parameters of input channel
     */
    rect_2.x = ALIGN16_DOWN(300);
    rect_2.y = 1080 - 700;
    rect_2.w = 700;
    rect_2.h = 700;
    stInputChnAttr.s32IsFreeRun = TRUE;
    stInputChnAttr.u32OutHeight = rect_2.h;
    stInputChnAttr.u32OutWidth = rect_2.w;
    stInputChnAttr.u32OutX = rect_2.x;
    stInputChnAttr.u32OutY = rect_2.y;
    /*
     *setting parameters of input channel<2>
     */
    MI_VDISP_SetInputChannelAttr(DevId, Chn_2_Id, &stInputChnAttr);

    //setting output color format
    stOutputPortAttr.ePixelFormat = E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420;
    //setting backgroud color of vdisp's output frame
    stOutputPortAttr.u32BgColor = YUYV_BLACK;
    //setting output framerate
```

```
    stOutputPortAttr.u32FrmRate = 30;
    //setting height of output frame
    stOutputPortAttr.u32Height = 1080;
    //setting width of output frame
    stOutputPortAttr.u32Width = 1920;
    //settin initial pts of output frame
    stOutputPortAttr.u64pts = 0;

    /*
     * setting parameters of output port<0>
     */
    MI_VDISP_SetOutputPortAttr(DevId,0,&stOutputPortAttr);

    /*
     * activating input channels <VDISP_OVERLAYINPUTCHNID,1,2>
     */
    MI_VDISP_EnableInputChannel(DevId, Chn_0_Id);
    MI_VDISP_EnableInputChannel(DevId, Chn_1_Id);
    MI_VDISP_EnableInputChannel(DevId, Chn_2_Id);

    /*
     * start vdisp device
     */
    MI_VDISP_StartDev(DevId);
}


/*
 * disable all channels enabled
 *      v
 * stop vdisp device
 *      v
 * close vdisp device
 *      v
 * exit vdisp
 */
void destruct_vdisp_module(void)
{
    MI_VDISP_DEV DevId = 0;
    MI_VDISP_CHN Chn_0_Id = VDISP_OVERLAYINPUTCHNID;
    MI_VDISP_CHN Chn_1_Id = 1;
    MI_VDISP_CHN Chn_2_Id = 2;
   /*
    * diable vdisp channels <VDISP_OVERLAYINPUTCHNID,1,2>
    */
    MI_VDISP_DisableInputChannel(DevId, Chn_0_Id);
```

```
    MI_VDISP_DisableInputChannel(DevId, Chn_1_Id);
    MI_VDISP_DisableInputChannel(DevId, Chn_2_Id);

   /*
    *stop vdisp device
    */
    MI_VDISP_StopDev(DevId);
   /*
    *close vdisp device
    */
    MI_VDISP_CloseDevice(DevId);
   /*
    *exit vdisp
    */
    MI_VDISP_Exit();
}

/*
 * binding vdec out0 -> vdisp inOverlay---\
 * binding vdec out1 -> vdisp in1 --------->vdisp out0--> disp in0
 * binding vdec out2 -> vdisp in2---------/
 */
void bind_disp_vdisp_vdec(void)
{
    MI_SYS_ChnPort_t stSrcChnPort;
    MI_SYS_ChnPort_t stDstChnPort;

    /*
     * binding vdisp out0-> disp in0
     * <chn,dev,port> : (0,0,0) -> (0,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_DISP;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

    /*
     * binding vdec out0-> vdisp inOverlay
     * <chn,dev,port> : (0,0,0) -> (VDISP_OVERLAYINPUTCHNID,0,0)
     */
```

```
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = VDISP_OVERLAYINPUTCHNID;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

    /*
     * binding vdec out0-> vdisp in1
     * <chn,dev,port> : (1,0,0) -> (1,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 1;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 1;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

    /*
     * binding vdec out2-> vdisp in2
     * <chn,dev,port> : (2,0,0) -> (2,0,0)
     */
    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 2;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 2;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_BindChnPort(&stSrcChnPort, &stDstChnPort, 30, 30);

}

void unbind_disp_vdisp_vdec(void)
{
    MI_SYS_ChnPort_t stSrcChnPort;
```

```
    MI_SYS_ChnPort_t stDstChnPort;

    stSrcChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_DISP;
    stDstChnPort.u32ChnId = 0;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_UnBindChnPort(&stSrcChnPort, &stDstChnPort);

    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 0;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = VDISP_OVERLAYINPUTCHNID;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_UnBindChnPort(&stSrcChnPort, &stDstChnPort);

    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 1;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 1;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_UnBindChnPort(&stSrcChnPort, &stDstChnPort);

    stSrcChnPort.eModId = E_MI_MODULE_ID_VDEC;
    stSrcChnPort.u32ChnId = 2;
    stSrcChnPort.u32DevId = 0;
    stSrcChnPort.u32PortId = 0;

    stDstChnPort.eModId = E_MI_MODULE_ID_VDISP;
    stDstChnPort.u32ChnId = 2;
    stDstChnPort.u32DevId = 0;
    stDstChnPort.u32PortId = 0;
    MI_SYS_UnBindChnPort(&stSrcChnPort, &stDstChnPort);
```

```
}

int main(void)
{
    signal(SIGINT, intHandler);

    MI_SYS_Init();
    construct_disp_module();
    construct_vdisp_module();
    construct_vdec_module();
    bind_disp_vdisp_vdec();
    while(keepRunning)
    {
        sleep(1);
    }
    unbind_disp_vdisp_vdec();
    destruct_disp_module();
    destruct_vdisp_module();
    destruct_vdec_module();
    MI_SYS_Exit();
    return 0;
}
```

7. PROCFS INTRODUCTION

7.1. cat

- Debug info

```
# cat proc/mi_modules/mi_vdisp/mi_vdisp0

==============================Private Vdisp0 Info
==============================
DevStatus
  Start
---------------------------------------------------- Input Port Info ----------------------------------------
PortID  PortStatus  ChnX  ChnY  ChnW  ChnH  IsFreeRun    TryOk      RecvOk
   0    Enabled     0     0     960   540   1     245356299    313332
   1    Enabled     960   0     960   540   1     355670297    313332
   2    Enabled     0     540   960   540   1     578760014    313331
   3    Enabled     960   540   960   540   1     757579130    313330
---------------------------------------------------- Output Port Info ----------------------------------------
Inited  FrmInterval  BgColor  PixelFmt  FrmRate  Width  Height      SendOk
  1     33333        8388736   0        30       1920   1080        471418
```

- Debug info analysis

  Record VDISP current usage status, device attribute, Input port attribute and Output port attribute, which can be dynamically got to debug and test.

- Parameter Description

Parameter		Description
device info	DevStatus	Working status of Vdisp: Uninit/ Init/ Start/ Stop
layer info	PortID	Inport ID Value range: [0~16], 16 is the overlay channel (PIP channel).
	PortStatus	Port status: Uninit/ Init/ Enabled/ Disabled
	ChnX	Input the X starting coordinate address of the inport Value range: [0~4094], need to be aligned according to the chip
	ChnY	Input the Y starting coordinate address of the inport Value range: [0~2159]
	ChnW	Input the image width of the inport, which needs to be aligned according to the chip Value range:[0~4096]

Parameter		Description
	ChnH	Input the image height of the inport Value range:[0~4096]
	IsFreeRun	Frame rate control or not 0: pts control play 1: free play
	TryOk	Try to get the times that the front-level bound port
	RecvOk	The times that get data from the front-level bound port successfully
Output port info	Inited	Output Port has been initialized or not 0: Uninit 1: Init
	FrmInterval	Output frame time interval, unit: US
	BgColor	Background color, printed here is decimal
	PixelFmt	Color space Value range: [0~E_MI_SYS_PIXEL_FRAME_FORMAT_MAX-1] 0-YUYV422, 9-YUVSP420
	FrmRate	Output frame rate
	Width	Image output width Value range: [0~4096], need to be aligned according to the chip
	Height	Image output height Value range:[0~2160], need to be aligned according to the chip
	SendOk	Frame statistics that successful pushed to the rear-level